eRIGHT Sign-In v4.1 — Go-live guide

For the administrator. Updated 19 October 2026 (late evening) for software **v4.1.0**, which is what is running on the server. Each step says what to do, where, what you should see, and how to prove it worked. Steps or paragraphs marked **v4.1** are new; everything else is unchanged from the v4.0 guide.

> **Nothing in this guide is assumed.** Every address, file path, command and current state below was read from the repository (`otherlands/SigninApp`), from the live server, or from DNS on 19 October 2026. Where a step happens in a vendor's console (GoDaddy, UniFi, Microsoft Entra, Teams) the guide tells you *which record or setting* is needed; exact menu names in those consoles were not checked and may differ from what is written here. If something does not match what you see, stop and say so.

What is already live

Piece	State on 19 Oct 2026, 23:50
Server	**v4.1.0 live** since 23:48 (22:48 UTC), database schema v11, all 69 tests passed on the server before it restarted, every health check green. Upgraded with the v4.1 upgrade script; the pre-upgrade database is `/root/signin-pre-upgrade-20261019T224833Z.sqlite` and the previous settings file `/root/eright-signin.env.pre-v41`. On Proxmox CT 106 (`signin`, Ubuntu 24.04 LXC) at `http://192.168.101.102:3000`; service `eright-signin`; code in `/opt/eright-signin`; settings in `/etc/eright-signin.env` (root only). All health checks green.
Register	8 active staff, 2 with cards. Company name and fire notice set. 16 roll calls recorded.
Backups	**on**: nightly 02:00 into `/opt/eright-signin/data/backups/`, 30 kept. Pre-upgrade copies in `/root/`. **v4.1: off-box copy on** — every backup is also pushed to platform-a (see "Off-box backup" below); first push 22:48 UTC. **Restore drill passed** 22:48 UTC (integrity ok, schema 11, no drift); it repeats weekly.
Teams cards	**configured**; last post succeeded 19 Oct 2026, 21:19 UTC (read from the live health check). The README notes this webhook's URL was pasted into a chat on 30 Sep and should be regenerated (step 1).
Retention	**6 years** (`RETENTION_DAYS=2190`), first run touched nothing.
Register assistant	**on**, using fast-a (`192.168.102.120:11434`, model Qwen2.5-Coder-7B). Admin only.
Public name / HTTPS	**not live**: `signin.eright.co.uk` does not exist in public DNS; the office LAN DNS answers `192.168.101.168`, a host that does not exist. Caddy config is ready but not installed (step 6).
SharePoint / Microsoft List	**not configured**: all `SP_*` values empty except the folder name (step 7).
Device tokens	**none issued**; devices use the shared `API_TOKEN` (step 3).

Back-door reader (ESP32)	Running v3.6 firmware (flashed 26 Sep). v4.1 firmware compiled, **not flashed** (step 4).
Front-door kiosk (Pi)	Parts ordered 26 Sep; **not built** (step 5).
Presence (phone / key-tag)	off (v4.1: off unless `PRESENCE=on`); no UniFi key, no tags, no anchor (step 8, optional).
Monitoring on platform-a	**deploy owed** — needs your sudo password on platform-a (step 9). Outside heartbeat **not chosen** (step 9b). Ops webhook **not created** (step 1b).
Trusted networks	**not set**: every LAN address, including the IoT Wi-Fi, still gets the full app on `:3000` (step 6g).
Code and updates	The server pulls updates over a **read-only deploy key** (GitHub: "CT 106 signin (read-only pull)"); `main` is protected against force-push and deletion. The repository is still **public** (see "Repository" below).

Off-box backup (v4.1, done for you)

After every backup the server copies it with `rsync` over SSH to **platform-a** (`192.168.102.100`), into `/home/aiadmin/signin-backups/`, as `signin-Mon.sqlite` … `signin-Sun.sqlite` (seven rolling copies). The key it uses lives in `/opt/eright-signin/data/.ssh/backup_push`; on platform-a that key is restricted in `~aiadmin/.ssh/authorized_keys` to `rrsync -wo` into that one directory — it can write backups there and do nothing else (a shell attempt was refused when tested). platform-a's host key is pinned in `/opt/eright-signin/data/.ssh/known_hosts`, taken from platform-a itself.

> **Honest limit:** platform-a is another machine on the same site. This protects against CT 106's disk or the Proxmox host failing; it does not protect against losing the building. That is what step 7 (SharePoint) adds. To restore from platform-a: `scp aiadmin@192.168.102.100:signin-backups/signin-<Day>.sqlite /root/` on the server, then `signinctl verify` and `signinctl restore` as in step 11.

Repository (v4.1)

The code is at `github.com/otherlands/SigninApp` and is **public**. It contains no passwords or tokens (those are only in `/etc/eright-signin.env` and the device `config.h`, both outside the repository), but it does describe the network: addresses, host names and how the fire roll call works. Making it private is your call and your click:

1. GitHub → the repository → **Settings** → **General** → bottom of the page, **Change repository visibility** → Private.
2. The server keeps updating: it already pulls with its own read-only deploy key.
3. **Before you do it**, note: the Pi kiosk installer (step 5) clones over plain https, which stops working for a private repository. Either build the kiosk first, or give the Pi its own read-only deploy key the same way. GitHub Actions on a private repository uses the account's included

minutes.

Order of work

1. Regenerate the Teams webhook; add the ops webhook (1b)
2. Staff, cards and phones
3. Device tokens
4. Flash the back-door reader
5. Build the front-door kiosk
6. Make `signin.eright.co.uk` real and switch on HTTPS
7. Entra app registration → SharePoint files + Microsoft List
8. Presence (optional)
9. Monitoring on platform-a; outside heartbeat (9b)
10. Run a drill
11. Day-to-day operation
12. Go-live checklist

Steps 1–5 need nothing outside the building and can be done today. Step 6 needs the domain and the router. Step 7 needs a Microsoft 365 administrator. Step 10 is the proof.

Things you will need

- SSH to the server as root: `ssh root@192.168.101.102` (key-based access already works from the development PC). Seen on 19 Oct: after a handful of SSH connections in a few minutes from the development PC, new connections to CT 106 *and* platform-a time out for 5–25 minutes, while the web app keeps answering. CT 106 has no firewall of its own (ufw inactive, no fail2ban), so the likely cause is the UniFi gateway's intrusion prevention; check its threat log if it gets in your way. Batch your commands into one session.
- The admin PIN: `grep ^ADMIN_PIN /etc/eright-signin.env` on the server. Never put it in a chat or e-mail.
- Access to: the Teams channel (owner), GoDaddy DNS for `eright.co.uk`, the UniFi controller, a Microsoft 365 Global/Application Administrator (step 7 only).
- For step 4: the development PC with PlatformIO, the ESP32 board and its CH343 UART lead.

Step 1 — Regenerate the Teams webhook

Why: the README records that the current webhook URL was pasted into a chat on 30 Sep 2026. Anyone with that URL can post into the channel. The cards work (last post 1 Oct), so this is hygiene, not repair.

1. In Teams, open the channel that receives the roll-call cards → ⋯ → **Workflows**. Find the flow named for the sign-in (it was created from "*Post to a channel when a webhook request is received*"), delete it, and create it again the same way. Copy the new HTTPS URL. (If the ⋯ menu has no Workflows entry, the README section "Teams channel" describes the Power Automate template route and a known gotcha with its "For each" step.)
2. On the server:

```
ssh root@192.168.101.102
nano /etc/eright-signin.env        # replace the value of TEAMS_WEBHOOK_URL=
node /opt/eright-signin/server.js --check-config
systemctl restart eright-signin
```

3. **Prove it:** open `http://192.168.101.102:3000/fire` on a phone, press **Start a DRILL**, confirm. Within seconds a card headed 🔴 **FIRE DRILL — ROLL CALL STARTED** appears in the channel with two buttons, **I'm SAFE** and **Open the roll call**. Press **END ROLL CALL**; a **DRILL ENDED** card follows. `Admin → Off-site copies` shows "Teams channel: ok".

New in v4.1 — the chaser: while anyone is still unaccounted for, the not-accounted names are re-posted to the same channel every 3 minutes (at most 10 times). `CHASER_MINUTES=0` in the env file switches it off; `CHASER_MAX` changes the limit.

1b. A second webhook for operational alerts (new in v4.1)

Why: the server now watches things outside itself (DNS for the public name, the certificate, the SharePoint secret's expiry, readers that have gone silent) and says so *once* when one goes wrong and once when it recovers. Those messages should not land in the fire channel.

1. In Teams, pick (or create) a channel for IT/operations. Create a Workflows webhook there exactly as in step 1. Copy its URL.
2. On the server: `nano /etc/eright-signin.env`, add `OPS_TEAMS_WEBHOOK_URL=<the new URL>`, then `node /opt/eright-signin/server.js --check-config` and `systemctl restart eright-signin`.
3. **Prove it:** Admin's health block lists an `ops-channel` line after the first alert. To force one, see step 9b (a heartbeat URL that does not answer produces a failing check).

Step 2 — Staff, cards and phones

2a. Staff names

Open `http://192.168.101.102:3000/admin`, enter the PIN (you stay logged in for 12 hours on that browser). Under **Staff**, check the 8 names. Add any missing with the "New staff name" box. Remove leavers with **Remove** (history is kept; their card is freed).

> Do not swipe a card while the cursor is in the "New staff name" box: the reader types the card number as a name. The server now refuses names that look like card numbers, but the swipe is still wasted.

2b. Assign cards (2 of 8 done)

Two ways; both end with the card's number in the person's **card UID** box.

1. **From a door tap:** tap the unassigned card on the back-door reader. The reader's LED flashes orange (unknown card). In Admin the line **Last unknown card seen** shows its number. On the right person's row press **Give last tapped card** (new in v4.1) and confirm. Tap the card again: green flash, kiosk shows them signed in.
2. **From a USB reader on the admin PC:** click into the person's card box and present the card; the USB-203 types the number and Enter. The server converts the 10-digit form to the same hex the door reader uses, so one card is one person at both doors.

To remove a card, type `CLEAR` in the box and press Enter.

2c. Phones

Every member of staff, on their phone, on the office Wi-Fi:

- Open `http://192.168.101.102:3000/fire` and add it to the home screen ("Add to Home Screen" in the browser's share/menu). This is the roll-call page; there is no designated marshal, so everyone has it.
- **Their personal "I'm SAFE" link.** In Admin → Staff, press **I'm SAFE link** on the person's row: the link (`…/safe#k=…`) is copied to the clipboard and shown. Send it to that person privately (the key in it can only ever mark *them* safe, nothing else; if a phone is lost press the button again and the old link stops working). They open it once on their phone — the page shows their name — then add it to the home screen. The Teams card's **I'm SAFE** button opens the same page and works on any phone that has opened its link once in that browser.
  v4.1: the key is after the `#`. A browser never sends that part to any server, so the key is in no server or proxy log, and it stays in the address bar so the home-screen icon keeps it. Links issued before v4.1 had `?k=`; none had been issued on the live server, but if any were, reissue them.

> Until step 6 is done these addresses only work on the office Wi-Fi. On mobile data the pages say "server unreachable" and tell the person to report to whoever is running the roll call in person. That is the honest state today. Issue the links *after* step 6e if you can, so they already carry the https address; links issued before carry the LAN address and must be reissued.

Print a QR code to `http://192.168.101.102:3000/fire` for the assembly-point sign (after step 6, reprint it with the https name).

> **Check in the first drill (not yet proven):** on an **iPhone**, a page added to the home screen does not share the Safari browser's storage. v4.1 keeps the key in the link itself for exactly this reason, so the home-screen icon should open with the person's name. Confirm it on at least one iPhone and one Android phone: open the home-screen icon with a drill running — it must show the name and the I'M SAFE button. If it says "no I'm SAFE link", note the phone model and report it.

Step 3 — Device tokens

Why: today every device shares one `API_TOKEN`. With per-device tokens a lost or stolen reader can be cut off alone, each device has only the permissions it needs, and the log names the device from its token rather than from what it claims.

In Admin → **Devices & tokens**, or on the server with `signinctl`:

```
cd /opt/eright-signin
node bin/signinctl.js token back-door-reader sign,fire      # the ESP32: card taps + the fi
re button (+ its heartbeat)
node bin/signinctl.js token door-kiosk sign                 # the Pi kiosk's USB card reade
r
```

A reader's heartbeat (v4.1, every 5 minutes) is accepted with any valid device token; no extra scope is needed. The Pi BLE scanner was removed in v4.1.

Each token is printed **once**. Put it straight into the device it is for (steps 4 and 5). Keep a note of which device has which name; never store the token itself anywhere else.

The shared `API_TOKEN` keeps working alongside. Once every device has its own token you may blank `API_TOKEN=` in the env file and restart — but only then, or the devices still using it stop being accepted.

Prove it: Admin → Devices shows the device with "never used" until its first request, then "last used …". `Admin → Log` shows `device token issued`.

Step 4 — Flash the back-door reader

Board: ESP32-S3 DevKitC-class, device name `back-door-reader`, currently on v3.6 firmware (flashed 26 Sep; cards proven 27 Sep). **v4.1** (compiled 19 Oct, not yet flashed) moves every network call into a background task, so a slow or dead server can no longer make the fire button or the card reader unresponsive, and sends a heartbeat every 5 minutes so the server notices if the reader goes silent. The v4 firmware adds: fire button retries until the server confirms (LED blinks red until then); card taps made while the server is unreachable are kept — through a power cut — and sent later; a tap restored after a power cut is recorded as "card tapped while reader was offline" and never changes the register (a lost tap over-counts, which is the safe failure).

1. Plug the board into the development PC with the **CH343 UART lead** (last time it appeared as `COM42`; check Device Manager → Ports for the current number).
2. On the PC:

```
cd "D:\eRIGHT\APPS\Access Lgging\V1\firmware\door-reader"
notepad include\config.h      # set API_TOKEN to the back-door-reader token from step
3 (keep WIFI_*, SERVER_URL, DEVICE_NAME as they are)
                              # v4.1: set  #define BLE_SCAN 0  (your config.h still sa
ys 1; keep 1 only if you will use key tags, step 8)
& "$env:USERPROFILE\.platformio\penv\Scripts\pio.exe" run -t upload --upload-port COM4
2
python tools\listen_door_reader.py COM42 40
```

   `config.h` is git-ignored and already holds the Wi-Fi name, password and server address.
3. **What a healthy boot prints** within 30 s: `[wifi] connecting to IOT` → `[server] http://192.168.101.102:3000/api/health -> 200` → `[nfc] PN5xx firmware … ready` → a `[status]` line every 30 s with `nfc=ready`.
4. **LEDs** (built-in RGB):

LED	Meaning
purple	connecting to Wi-Fi / Wi-Fi lost
dim blue	ready, waiting for a card
green flash	card accepted: person signed in or out
orange flash	card not confirmed: unknown card or wrong token (refused), or server unreachable (tap **kept**, sent when the server answers). v4.1: with the server unreachable, every further tap flashes orange at once — **do not tap again**; a second tap would later count as signing back out.
red 3 s	roll call started from the button and **confirmed by the server**
red blinking	button held, server not yet confirmed; retries every 3 s for up to 15 min. **Start the roll call from a phone as well.**
orange steady at boot	PN532 not found: see `firmware/door-reader/WIRING.md` step 6

5. **Prove it:** (a) tap an assigned card → green flash, kiosk changes; (b) hold the button 2 s → red, `/fire` shows a roll call "by door button", end it from a phone; (c) in Admin → Log the events show device `back-door-reader`; (d) the serial log shows `[http] /api/heartbeat -> 200` within 5 minutes.
6. **Then tell the server to expect it** (v4.1): add `EXPECT_DEVICES=back-door-reader` to `/etc/eright-signin.env`, `--check-config`, restart. From then on a reader silent for 15 minutes (`DEVICE_SILENT_MINUTES`) shows as a failing `drift:device:back-door-reader` check and, with step 1b done, posts once to the ops channel. Do this only *after* flashing: the v3.6 firmware sends no heartbeat and would be reported silent all day.
7. Fit it back on the wall (mounting and the fire-button label are in `WIRING.md`, "Mounting").

Step 5 — Build the front-door kiosk (Raspberry Pi)

Parts were ordered from The Pi Hut on 26 Sep 2026 (Pi 5 2 GB, Argon Industria HMI 10CS display + enclosure, Argon PoE+ HAT, THRML cooler, 32 GB SD). The build is written up in the README section **"Door kiosk — Raspberry Pi build"** and the tick-box bench card `docs/pi-kiosk-bench-card.md`. In short:

1. Flash the SD with Raspberry Pi Imager: **Raspberry Pi OS Lite (64-bit)**, hostname `door-kiosk`, a user, **SSH enabled**, locale Europe/London.
2. Assemble: THRML cooler on the Pi (leave the case's passive column out), PoE+ HAT on the header, screen power lead to the HAT's 5 V header, screen ribbon to the DSI port, close the enclosure.
3. One Ethernet cable from a PoE port on the Corporate network. Give `door-kiosk` a DHCP reservation in UniFi.
4. From a PC: `ssh <user>@door-kiosk.local`, then

   ```
   git clone https://github.com/otherlands/SigninApp.git ~/SigninApp
   sudo KIOSK_URL=http://192.168.101.102:3000 KIOSK_CARD_TOKEN='<door-kiosk token from st
   ep 3>' bash ~/SigninApp/deploy/kiosk-pi/install.sh
   sudo reboot
   ```

   After the reboot the screen shows the sign-in page full-screen and never blanks.
5. Plug the USB-203 card reader into the Pi. Present an unassigned card: the page says "Card … is not assigned to anyone" and Admin shows it under **Last unknown card seen**. That one scan proves reader → page → token → server.

> None of the kiosk build has run on real hardware yet — the parts had not arrived when the installer was written. Expect to adjust the screen-rotation setting and possibly the Chromium package name; the bench card lists what to check. After step 6, re-run the installer with `KIOSK_URL=https://signin.eright.co.uk`.

Step 6 — Make `signin.eright.co.uk` real and switch on HTTPS

Why: three things need it. (1) Staff phones on mobile data can reach `/fire` and `/safe` at the assembly point. (2) The fire page's **offline copy** (it opens and shows the last register even with the server dead) only works over HTTPS. (3) The admin login cookie is sent encrypted.

Facts checked on 19 Oct 2026: `signin.eright.co.uk` has no public DNS record; `eright.co.uk`'s nameservers are GoDaddy's (`ns13` / `ns14.domaincontrol.com`); inside the office the UniFi gateway (192.168.101.1) answers `192.168.101.168` for that name and nothing exists at .168; the office's public address was `46.37.62.121`; nothing listens on ports 80/443 on the server.

> **What comes through from the internet — the one proxy order.** Caddy (installed in 6d) looks at where each request comes from. From the **office network** the name gives full access, same as the LAN address. From the **internet** exactly this is allowed and nothing else: the roll-call page (`/fire`), the "I'm SAFE" page (`/safe`), the files they load, reading the roll call (`/api/fire`, `/api/me`, the live stream) and **one write** — a person marking *themselves* safe with their personal key. Admin, kiosk, visitor pages, sign-in, start/end roll call, marshal ticks, metrics, export: all answer `403 Not available from outside the office`. The app checks the same list a second time, so a mistake in the proxy cannot open anything. Names of who is on site are still readable by anyone who finds the address: that is the residual exposure, and it is a read. This was proven on 19 Oct 2026 with a real Caddy against the live server: 12 allowed requests through, 26 refused. **v4.1:** outside the office the register shows names and marks only (no company, host, vehicle or badge); Caddy's request log has the personal key, the admin PIN and the device tokens removed — checked with Caddy v2.11.7 on the server: none of them appeared in its log.

6a. Public DNS (GoDaddy)

In GoDaddy's DNS management for `eright.co.uk`, add an **A record**: name `signin`, value the office's public IPv4 address (`46.37.62.121` as seen on 19 Oct — confirm it is static with the ISP; if it can change you need dynamic DNS or the DNS-01 variant in `deploy/Caddyfile`), TTL short (e.g. 600). Wait until `nslookup signin.eright.co.uk` from a phone on mobile data returns that address.

6b. Port forwarding (UniFi)

On the UniFi gateway, forward **TCP 80 and TCP 443** from the WAN to `192.168.101.102`. Port 80 is needed for the certificate challenge and for redirecting http to https; 443 is the site.

6c. LAN DNS (UniFi)

Change the local DNS record for `signin.eright.co.uk` from `192.168.101.168` to `192.168.101.102` so phones on the office Wi-Fi reach the server directly by name. Check from the server: `getent hosts signin.eright.co.uk` must print `192.168.101.102`.

6d. Caddy on the server

```
ssh root@192.168.101.102
cd /opt/eright-signin && git pull --ff-only
sudo bash deploy/install-caddy.sh
```

The script installs Caddy, puts `deploy/Caddyfile` in place (the allow-list above, proxying to the app on :3000), validates it and starts it. Caddy obtains the certificate itself over port 80. If it prints `000` for the https check, the name or the forward is not right yet: `journalctl -u caddy -n 30` says which.

Prove the boundary from a phone on mobile data: `https://signin.eright.co.uk/admin` must say "Not available from outside the office."; `/fire` must load read-only (no SAFE/MISSING buttons, a note saying so).

6e. Tell the app its public address

```
nano /etc/eright-signin.env        # PUBLIC_URL=https://signin.eright.co.uk
node /opt/eright-signin/server.js --check-config
systemctl restart eright-signin
```

From now on the Teams cards' buttons point at the https name. The plain `http://192.168.101.102:3000` keeps working for the kiosk and the printed QR codes.

6f. Let the server watch the name (v4.1)

Add `DRIFT_HOSTNAME=signin.eright.co.uk` (and, if the public address is static, `EXPECT_PUBLIC_IP=46.37.62.121` or whatever 6a used) to the env file, `--check-config`, restart. Every hour the server checks that the name resolves to itself on the office DNS, exists in public DNS, and that the certificate has at least 14 days left; a change posts once to the ops channel (step 1b). Set it only after 6a–6c: before then those checks fail by design.

6g. Trusted networks (v4.1)

Why: the server answers on `:3000` to every LAN address, including the IoT Wi-Fi the back-door reader uses. With `TRUSTED_NETS` set, an address outside those subnets gets only what the internet gets (fire page, names and marks, self-marking) plus the device routes *with a device token* — never Admin, the kiosk sign-in or marshal ticks. Requests through Caddy are judged on the real client address.

1. In the UniFi controller, list the networks and their subnets. The office/corporate network that CT 106 sits on is `192.168.101.0/24` (the server's own address). Decide which other networks staff PCs, the kiosk and phones use; the IoT network the reader uses should normally **not** be trusted (it has its token).
2. Env file: `TRUSTED_NETS=192.168.101.0/24,<other trusted subnets>`, `--check-config`, restart.
3. **Prove it:** from a device on an untrusted network, `http://192.168.101.102:3000/admin` must say "Not available from this network"; `/fire` must load read-only. A card tap on the reader must still be green.

6h. Prove it

- On a phone **with Wi-Fi off**: `https://signin.eright.co.uk/fire` loads with a padlock; below the buttons it says **offline copy: ready**.
- Turn the phone to flight mode, reopen the page from the home screen: it still opens and shows "SERVER UNREACHABLE — showing the last register this device saw…".
- `https://signin.eright.co.uk/safe` on a phone that has opened its personal link shows the person's name and, when a roll call is running, the I'M SAFE button. A phone that has not says so and tells them to ask for their link.
- Issue (or reissue) everyone's "I'm SAFE" link now so it carries the https address (step 2c); re-add `/fire` to home screens with the new address; reprint the assembly-point QR code.

Step 7 — Entra app registration → SharePoint files + Microsoft List

Why: with the building dark, the only copies of the register that survive are the ones already in Microsoft's cloud. The server pushes three files (`who-is-on-site.txt`, `roster.json`, `events-<day>.csv`) into a SharePoint folder within 15 s of every change, and (new in v4) keeps a **Microsoft List** with one row per person on site plus a status row, which staff can open — and edit — in the Teams or SharePoint app on their phones. Until this is done, **nothing leaves the building automatically**.

The full one-page sheet for the Microsoft 365 administrator is `docs/entra-app-registration-sheet.md`; it explains each permission and the exact Graph calls the server makes. Summary:

1. **Register the app** in the Entra admin centre: App registrations → New registration. Name `eRIGHT Sign-In Server`, this organisational directory only, no redirect URI. Copy the **Directory (tenant) ID** and **Application (client) ID**.
2. **Create a client secret** (Certificates & secrets → New client secret, 24 months). Copy the **Value** immediately; it is shown once. Put the expiry date in the diary. Type it only into the env file on the server — never into a chat.
3. **Permissions**: API permissions → Microsoft Graph → Application permissions → `Sites.Selected` (preferred; the app can touch nothing until step 4 grants one site). Grant admin consent.
4. **Grant the app write on the one site** via Graph Explorer (the sheet gives the two requests: GET the site id, then POST `/sites/<id>/permissions` with `roles: ["write"]` for the app). A `201 Created` is the proof. The List needs the same write permission as the files; no extra permission.
5. In that site's Documents library create the folder `eRIGHT Ltd/Sign-in`.
6. On the server:

```
nano /etc/eright-signin.env
  SP_TENANT_ID=<Directory (tenant) ID>
  SP_CLIENT_ID=<Application (client) ID>
  SP_CLIENT_SECRET=<secret Value>
  SP_SITE=eright.sharepoint.com:/sites/<SiteName>
  SP_DRIVE=                       (blank = the site's default Documents library)
  SP_FOLDER=eRIGHT Ltd/Sign-in    (already set)
  SP_LIST=Sign-in register        (the List is created on first sync if it does not exi
st)
node /opt/eright-signin/server.js --check-config
systemctl restart eright-signin
```

Record the secret expiry (v4.1): add `SP_SECRET_EXPIRES=<YYYY-MM-DD from step 2>` to the env file; from 30 days before it the `drift:sp-secret` check fails and the ops channel is told.

Ticking in the List (v4.1): the List has a **Tick** column (SAFE / MISSING). During a roll call, anyone with the List on their phone can set it on a person's row; within 30 s the server applies it as a mark ("<their name> (Microsoft List)") and empties the cell. Rules, for safety: MISSING always counts; SAFE counts

only for someone nobody else has marked; a SAFE that was already in the List when the roll call started is thrown away (tick again). The List only acts while the server is running and can reach Microsoft.

Prove it: sign someone in on the kiosk. Within 15 s, **Admin** → **Off-site copies** shows "SharePoint → … ok … (3 files)" and "Microsoft List "Sign-in register": ok (n rows)". On a phone, open the SharePoint folder and the List in the Teams app; put shortcuts to both on every staff phone. If it fails, the error text in Admin names the step: the sheet's last table maps each message to the step to redo.

> The Microsoft List code has been exercised only against a simulated Graph service, not against the real tenant: the app registration did not exist when it was written. The first real sync is the test. Report the exact error text if it fails.

Step 8 — Presence: phone and key-tag signals (optional)

v4.1: presence is off unless you add `PRESENCE=on` to the env file (the UniFi and scanner settings do nothing without it). Signs people **in** automatically when their phone joins the office Wi-Fi or their key tag is heard inside; may sign **out** only someone it signed in itself, and only when both signals agree and the tag walked out past a door; never overrules a tap. Everything else it only *shows* ("probably left"). The engine is tested with simulated controllers and scanners; it has **not** been run against the real UniFi controller or real tags.

8a. Phone signal (UniFi)

1. In the UniFi controller create a **read-only API key**.
2. Env file: `UNIFI_URL=https://<controller address>`, `UNIFI_API_KEY=<key>`, optionally `UNIFI_SSID=<office network name>`. `--check-config`, restart.
3. Staff set their phone's Wi-Fi "Private Address" for the office network to **Fixed** (not rotating), or the phone looks new every day.
4. Admin → **Presence — phones & key tags**: phones seen on the Wi-Fi in the last 10 minutes appear in a picker; assign each to its owner.

8b. Key-tag signal

Needs iBeacon-class tags on key rings, the back-door reader (already scans), an inside "anchor" ESP32 (same firmware, `SCANNER_ROLE "anchor"`, `HAS_PN532 0`), with `BLE_SCAN 1` in their `config.h` and a `presence`-scoped token (the Pi BLE scanner was removed in v4.1). Tags heard appear in the same Admin picker. None of this hardware has been bought yet.

Prove it: `signin_wifi_last_poll_ok 1` on `/metrics`; a staff member whose phone is enrolled shows 📱 on the kiosk when in.

Step 9 — Monitoring on platform-a

The estate's Prometheus/Alertmanager on platform-a (`192.168.102.100`) e-mails the team when a roll call starts, when someone is lone-working out of hours, when the server is down and when the off-site copy is failing. The rules and scrape job live in the `llm-cluster` repository; deployment to platform-a is still owed. From your PC:

```
ssh -t aiadmin@192.168.102.100 "cd ~/llm-cluster && git pull --ff-only && SIGNIN_ADDR=192.1
68.101.102 bash scripts/deploy-signin-monitoring.sh 2>&1 | tee ~/deploy-signin-monitoring.l
og"
```

Give CT 106 a **DHCP reservation** in UniFi first so `192.168.101.102` never moves.

The script needs **sudo on platform-a** (it edits `/etc/hosts` and the Prometheus configuration); `ssh -t` lets it ask for the password. That is why it was not run for you.

v4 added series worth alerting on: `signin_check_ok{check}` (0 = a health check failing — in v4.1 this includes `backup-offbox`, `restore-drill`, `heartbeat`, `llm` and every `drift:*`), `signin_backup_age_seconds`, `signin_job_failures{job}`, `signin_rollcall_drill`. Adding rules for them is a change in the llm-cluster repo, not here.

9b. An outside heartbeat (v4.1)

Why: Prometheus and the ops channel both run inside the building. If the Proxmox host, the switch or the power goes, nothing inside can report it. A heartbeat service outside does: the server calls its URL every 5 minutes, and the service e-mails or texts you when the calls stop.

1. Choose the service (your decision; none has been chosen). An example of the kind: healthchecks.io gives each check a ping URL like `https://hc-ping.com/<uuid>`; set its period to 5 minutes and grace to 10.
2. Env file: `HEARTBEAT_URL=<the ping URL>` (optionally `HEARTBEAT_MINUTES=5`), `--check-config`, restart.
3. **Prove it:** the service shows the first ping within 5 minutes; Admin shows "Heartbeat (outside watcher): ok". Then `systemctl stop eright-signin` for 15 minutes on a quiet day and check the alert arrives; start it again.

Prove it: run the step-10 drill again; the Alerts@ e-mail must arrive as well as the Teams card.

Step 10 — Run a drill

This is the proof that the whole chain works, and it leaves a record (time to all-accounted) for the fire risk assessment.

1. Pick a quiet time. Make sure at least three people are signed in (kiosk or cards).
2. On a phone, `/fire` → **Start a DRILL** → confirm. (Or on the server: `node bin/signinctl.js drill "your name"`.)
3. Watch: the Teams card arrives (🔴 FIRE DRILL — ROLL CALL STARTED) listing who is on the register; the kiosk shows the red banner; every open `/fire` page switches to the roll call within a second.
4. Ask two people to press **I'm SAFE** on the Teams card or their `/safe` shortcut (their phone must have opened its personal link once, step 2c). The roll-call page shows them ✓ SAFE "by <name> (self)" and the Safe counter says "(2 by phone)".
5. Tick the rest SAFE on the roll-call page with your name in the "Your name (marshal)" box. When the last one is ticked the page turns green, ALL n ACCOUNTED FOR, and a green card posts to Teams.
6. Press **END ROLL CALL**. The DRILL ENDED card gives the duration and "All accounted for after n min".
7. Admin → **Roll-call history**: the row says *drill*, shows self-marks and the time to all-accounted. Keep that for the fire log.

Repeat the drill after step 6 with phones on mobile data, and after step 7 to see the SharePoint files and the List change during it.

Step 11 — Day-to-day operation

Daily glance

Admin → **Off-site copies** opens with one line: **All health checks pass**, or which ones fail and why (v4.1). Below it: SharePoint, List, Teams, backups (local and off-box), the last restore drill, the heartbeat, the assistant, presence, retention and TLS. For every health check and job, on the server:

```
cd /opt/eright-signin
node bin/signinctl.js status          # version, every check, every job, backup age, warnin
gs
```

Anything other than `status: OK` names the failing check and why.

Forgotten sign-outs

Staff or visitors signed in before midnight show ⚠ on the kiosk and roll-call page and under Admin → **Forgotten sign-outs**. Close them there. "Close all" skips anyone whose phone or tag says they are inside right now.

Backups and restore

Nightly at 02:00 into `/opt/eright-signin/data/backups/` (30 kept; also copied to SharePoint once step 7 is done). **v4.1:** each backup is also pushed off this server (see "Off-box backup" on the cover page), and once a week the newest backup is restored into a scratch copy, integrity-checked and refolded — a failure shows as the `restore-drill` health check. Run it by hand with `curl -s -X POST -H "X-Admin-Pin: $PIN" localhost:3000/api/restore-drill/run`. To check a backup, or to restore one:

```
node bin/signinctl.js verify /opt/eright-signin/data/backups/<file>.sqlite     # integrity,
schema, counts
node bin/signinctl.js restore /opt/eright-signin/data/backups/<file>.sqlite --yes
# stops the service, copies the file in beside the live one, swaps only when the copy succe
eded, restarts, prints status.
# The replaced register is kept as signin.sqlite.replaced-<time>.
```

Do one restore drill on a quiet day so the procedure is known before it is needed.

Upgrading

```
bash /opt/eright-signin/deploy/upgrade-ct106.sh
# backs the register up to /root first, pulls, runs all tests, keeps the env file, restart
s, prints health before and after
# v4.1: a failing test = the checkout is put back and nothing restarts. If the new version
does not answer healthy
# with its new version number within 30 s, the previous code AND the pre-upgrade database g
o back automatically.
```

Before editing the env file, and after: `node server.js --check-config`. The server refuses to start on a bad value and says which.

Other commands

Command	Does
`node bin/signinctl.js people`	staff list: card/phone/tag, in or out
`node bin/signinctl.js token <name> <scopes>` / `revoke <name>`	device tokens (step 3)
`node bin/signinctl.js ask "…"`	ask the register (also in Admin → Ask the register)
`node bin/signinctl.js rebuild-projections`	refold "who is in" from the event log; reports whether anything differed (it should not)
`journalctl -u eright-signin -n 50 --no-pager`	server log
`curl -s localhost:3000/metrics | grep signin_`	everything Prometheus sees

Records to keep

- Where the admin PIN and the device-token names are written down (not the tokens).
- The client-secret expiry date from step 7 (24 months) — the SharePoint copy stops silently when it expires; Admin will show `AADSTS7000215`.
- Drill dates and times to all-accounted (Admin → Roll-call history).
- Retention decision: 6 years for visitor details, events and roll calls (presence sightings 14 days). Recorded in the README; change `RETENTION_DAYS` if the policy changes.

Step 12 — Go-live checklist

☐ Teams webhook regenerated; drill card received (step 1)

☐ All staff named in Admin; every card assigned; leavers removed (step 2)

☐ Every staff phone has `/fire` and its personal "I'm SAFE" link on the home screen; assembly-point QR printed (step 2)

☐ Device tokens issued for the reader and the kiosk; shared API_TOKEN blanked afterwards (step 3)

☐ Back-door reader on v4 firmware; card green, button red, both visible in Admin → Log (step 4)

☐ Front-door kiosk built, on the wall, card reader proven (step 5)

☐ `signin.eright.co.uk` resolves publicly and on the LAN to the server; https padlock; `PUBLIC_URL` updated; "offline copy: ready" on a phone; `/admin` from mobile data says "Not available from outside the office." (step 6)

☐ SharePoint files and Microsoft List updating within 15 s; shortcuts on phones; secret expiry in the diary (step 7)

☐ Presence configured, or consciously left off (step 8)

☐ Monitoring deployed on platform-a; alert e-mail received during a drill (step 9)

☐ A full drill done with phones on mobile data and recorded in Roll-call history (step 10)

☐ One restore drill done from a backup (step 11); the weekly automatic one shows ok in Admin

☐ Ops webhook set (step 1b); outside heartbeat chosen and set (step 9b)

☐ `TRUSTED_NETS` set (step 6g); `DRIFT_HOSTNAME` set after the name works (step 6f); `EXPECT_DEVICES` set after the reader is reflashed (step 4); `SP_SECRET_EXPIRES` set (step 7)

☐ An iPhone and an Android home-screen "I'm SAFE" icon proven in a drill (step 2c)

☐ Repository made private (cover page, "Repository")

☐ CT 106 has a DHCP reservation; a UPS covers the Proxmox host, switch, access point and router (README, "What actually survives the fire today")

Still true after all of this

The register is correct up to the instant the server, its switch or its power goes; from then on the SharePoint files and the Microsoft List are the record, readable on any phone. Self-marking and the roll-call page keep working only while the server can be reached (LAN, or the internet via step 6). Ticking people off in the List (v4.1) works only while the server is running and can reach Microsoft; with the server gone, a tick in the List is visible to everyone reading the List but is applied by nobody. The fire-warden headcount at the assembly point remains the control of record.

Source of truth for everything above: the repository README ("v4.1", "v4.0", "Install on Ubuntu", "Teams channel", "SharePoint off-site copy", "Door kiosk", "ESP32 door reader"), `docs/entra-app-registration-sheet.md`, `docs/pi-kiosk-bench-card.md`, `deploy/*.sh`, `deploy/Caddyfile`, and the live server on 19 October 2026.